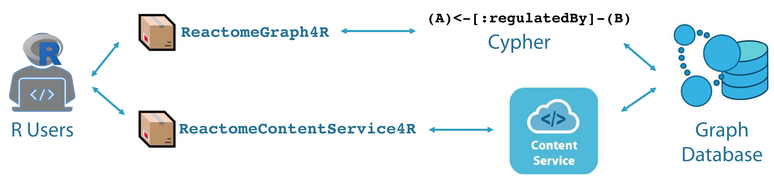
R interface to the Reactome Graph Database

Chi-Lam Poon[1], Justin Cook[2], Solomon Shorser[3], Joel Weiser[3], Robin Haw[3] and Lincoln Stein[3]

[1]Weill Cornell Medicine, New York, NY, USA; [2]Simon Fraser University, Burnaby, BC, Canada; [3]Ontario Institute for Cancer Research, Toronto, ON, Canada

Introduction

Reactome is a knowledgebase of biomolecular pathways. As of the current release, its holdings comprise 13,827 human reactions organized into 2,536 pathways involving 11,084 proteins, 1,857 small molecules, and 432 drugs, with the support of 33,752 literature references. These molecular entity and event annotations are systematically represented as an ordered network of molecular reactions, naturally forming a graph of biological knowledge. Reactome has migrated from a relational database to a Neo4j graph database for efficient storage, processing, and access with these highly interconnected data. To minimize the usage of Neo4j Cypher, we developed an interface, ReactomeGraph4R, for interacting between R and the Reactome graph database. We developed a second R package, ReactomeContentService4R, to wrap the RESTful API of the Content Service built on top of the graph core. These packages are available through Bioconductor and fulfill the needs of omics analyses with Reactome knowledge.



Fetching Reactome Instances

Basically, `ReactomeGraph4R` retrieves data with a **network structure** by translating R commands to Cypher queries and communicating with a local Graph Database. `ReactomeContentService4R` retrieves **specific bits of information** through constructing and sending http requests to the Content Service. Returned data are formatted as R objects in the two packages.

For example, to fetch details of a protein "misfolded CFTR [endoplasmic reticulum membrane]" with standard id `R-HSA-8868730` (a common functionality in both packages):

- `ReactomeGraph4R` function:

```
matchObject(id = "R-HSA-8868730")
#> $databaseObject
#>                    schemaClass  speciesName startCoordinate isInDisease
#> 1 EntityWithAccessionedSequence Homo sapiens               1       FALSE
#>                                   displayName     stIdVersion    dbId
#> 1 misfolded CFTR [endoplasmic reticulum membrane] R-HSA-8868730.2 8868730
#>                                                                      name
#> 1 misfolded CFTR, Cystic fibrosis transmembrane conductance regulator, CFTR_HUMAN
#>          referenceType endCoordinate          stId
#> 1 ReferenceGeneProduct          1480 R-HSA-8868730
```

- Cypher query:

```
MATCH (dbo:DatabaseObject)
WHERE dbo.stId = "R-HSA-8868730"
RETURN dbo
```

- `ReactomeContentService4R` function:

```
query(id = "R-HSA-8868730")
#> $dbId
#> [1] 8868730
#>
#> $displayName
#> [1] "misfolded CFTR [endoplasmic reticulum membrane]"
#>
#> $stId
#> [1] "R-HSA-8868730"
#>
#> $stIdVersion
#> [1] "R-HSA-8868730.2"
… [truncated]
```

- Content Service query:

```
curl -X GET "https://reactome.org/ContentService/data/query/enhanced/R-HSA-8868730"
-H "accept: application/json"
```

*Snippets for R code were generated by the reprex package (v2.0.0)

Fetching preceding/following Reactions

Events are building blocks in the database and they have two subclasses, Pathways and ReactionLikeEvents. Pathways are ordered groups of Reations/ReactionLikeEvents. `ReactomeGraph4R::matchPrecedingAndFollowingEvents()` can get preceding and following Reactions of a Reaction of interest via the "precedingEvent" relationship.

- In `ReactomeGraph4R`, data can be returned as either "row" or "graph" format: "row" are mostly dataframes, and "graph" contains `nodes` and `relationships`, which can be used for network visualization

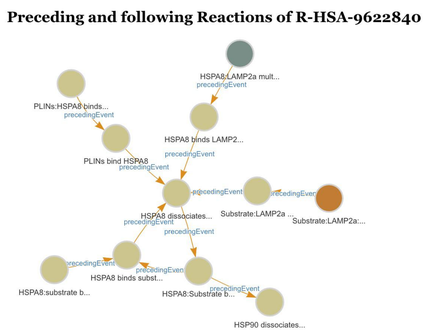
```
row <- matchPrecedingAndFollowingEvents(event.id = "R-HSA-9622840",
                                        type = "row", depth = 2)
str(row, max.level = 1)
#> List of 4
#>  $ precedingEvent:'data.frame':  3 obs. of  12 variables:
#>  $ event         :'data.frame':  1 obs. of  12 variables:
#>  $ followingEvent:'data.frame':  9 obs. of  12 variables:
#>  $ relationships :'data.frame':  11 obs. of  9 variables:
```

- Get a "graph" object, and then process it to an object of a visualization package

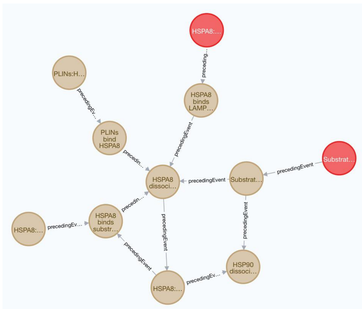
```
graph <- matchPrecedingAndFollowingEvents(event.id = "R-HSA-9622840",
                                          type = "graph", depth = 2)

# Further steps for plotting...
```

`visNetwork` visualization in R



Neo4j graph



Finding Referrals

PhysicalEntities are participants of Reactions (Pathway > Reaction > PhysicalEntity). Using `ReactomeGraph4R::matchReferrals()` we can get Referrals of a given object (= find instances in a reverse direction). For instance, to **enumerate** the Reactions that have DefinedSets (a PhysicalEntity subgroup):

```
# Fetch all human DefinedSet objects
defined.sets <- matchObject(schemaClass = "DefinedSet", species = "human")
defined.sets <- defined.sets[["databaseObject"]]

# Get Referrals for each DefinedSet
reactions <- lapply(defined.sets$stId, function(id) {
  referrals <- matchReferrals(id, type = "row")
  # further processing...
})
reactions <- data.table::rbindlist(reactions, fill = TRUE)
head(reactions, 3)
```

schemaClass <chr>	speciesName <chr>	isInDisease <lgl>	displayName <chr>	stIdVersion <chr>
Reaction	Homo sapiens	FALSE	HSP90 dissociates from LAMP2a	R-HSA-9626276.2
Reaction	Homo sapiens	FALSE	Substrate:LAMP2a binds HSP90	R-HSA-9622831.3
Reaction	Homo sapiens	FALSE	PSMD14 cleaves K63-linked ubiquitin	R-HSA-5691431.2

3 rows | 1-5 of 20 columns

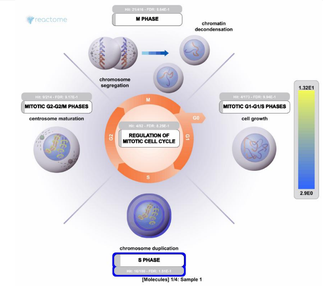
peName <chr>	peStId <chr>	peDbId <int>	peType <chr>	maxUnitCount <int>
HSP90AA1, HSP90AB1 [lysosomal lumen]	R-HSA-9622845	9622845	output	NA
HSP90AA1, HSP90AB1 [lysosomal lumen]	R-HSA-9622845	9622845	input	NA
K63-Ub [cytosol]	R-HSA-450143	450143	output	NA

3 rows | 13-17 of 20 columns

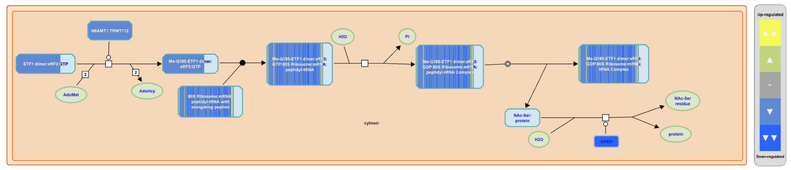
Fold change visualization

The Diagram Exporter in `ReactomeContentService4R` can overlay results from the `Analysis Service` on top of a diagram with the unique token associated with the performed analysis.

```
# Animated GIF for EHLD
exportImage(id = "R-HSA-69278", sel = "R-HSA-69242", output = "diagram",
            format = "gif", token = token)
```



```
exportImage(id = "R-HSA-72764", output = "diagram", format = "png",
            token = token, quality = 10)
```



Mapping external ids

Non-Reactome identifiers, e.g. UniProt ids, gene symbols, GO terms, associated with an Event (Pathway or Reaction) can be retrieved by `ReactomeContentService4R::event2Ids()`.

```
ids <- event2Ids(event.id = "R-HSA-72764")
head(ids$geneSymbol)
#> [1] "RPL29"  "RPL28"  "RPL27A" "RPL27"  "RPL3L"  "RPL3"
head(ids$primaryIdentifier)
#>    dbId           displayName
#> 1 72445  UniProt:P47914 RPL29
#> 2 72443  UniProt:P46779 RPL28
#> 3 72441 UniProt:P46776 RPL27A
#> 4 72439  UniProt:P61353 RPL27
#> 5 72449  UniProt:Q92901 RPL3L
#> 6 72447   UniProt:P39023 RPL3
```

- Note that there may be multiple symbols for one gene, it's better to map to primary identifiers such as UniProt ids

Code

- https://github.com/reactome/ReactomeGraph4R
- https://github.com/reactome/ReactomeContentService4R

Acknowledgements

This work was funded by Google Summer of Code, 2020. The development of Reactome is supported by grants from the US National Institutes of Health (U41 HG003751) and the European Molecular Biology Laboratory. We thank the event support made possible by the fellowship from Open Bioinformatics Foundation (OBF).